Data Mining & Business Intelligence (IT647)
Selected Topics In Data Science (IT688)

Information Technology Department
College Of Computer
Qassim University

DR.MONA ALSALEEM

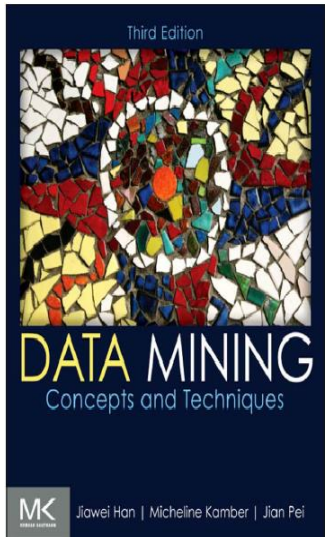
Data Mining & Business Intelligence Course_IT674

Data Mining in BI

— Chapter 8 —

Classification: Basic Concepts and Techniques

Text Book: “Data Mining: Concepts and Techniques”, Third Edition



Outlines

- Classification: Basic Concepts 
- Decision Tree Induction
- Bayes Classification Methods
- Rule-Based Classification
- Model Evaluation and Selection
- Summary

Supervised vs. Unsupervised Learning

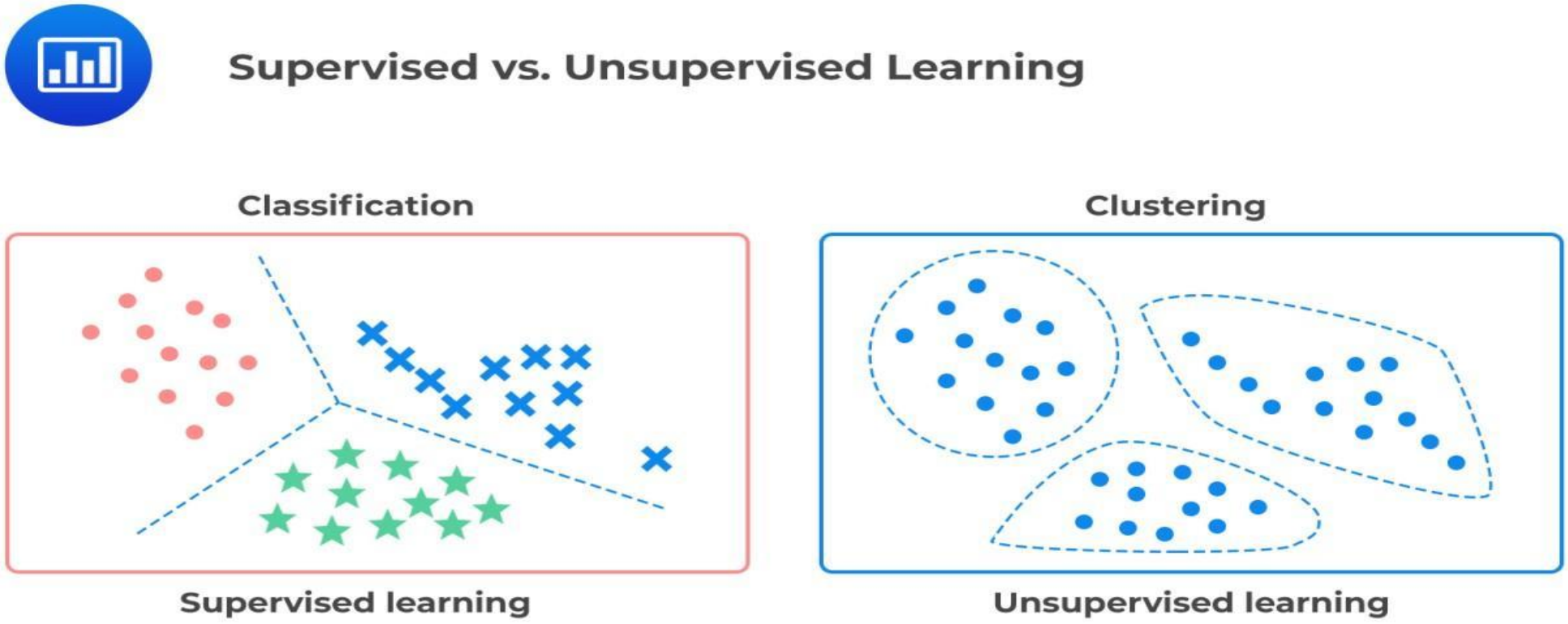
- **Supervised learning (classification)**
  - Supervision: The training data (observations, measurements, etc.) are accompanied by labels indicating the class of the observations
  - New data is classified based on the training set.

- **Unsupervised learning (clustering)**
  - The class labels of training data is unknown
  - Given a set of measurements, observations, etc. with the aim of establishing the existence of classes or clusters in the data

What Is Classification?

- Classification is a form of data analysis that extracts models describing important data classes.
- Given a collection of records (training set)
  - Each record is characterized by a tuple (x, y) where x is the attribute set and y is the class label
    - ***x***: attribute, predictor, independent variable, input
    - ***y***: class, response, dependent variable, output
- Task:
  - Learn a model that maps each attribute set ***x*** into one of the predefined class labels ***y***

What Is Classification?



General Approach to Classification

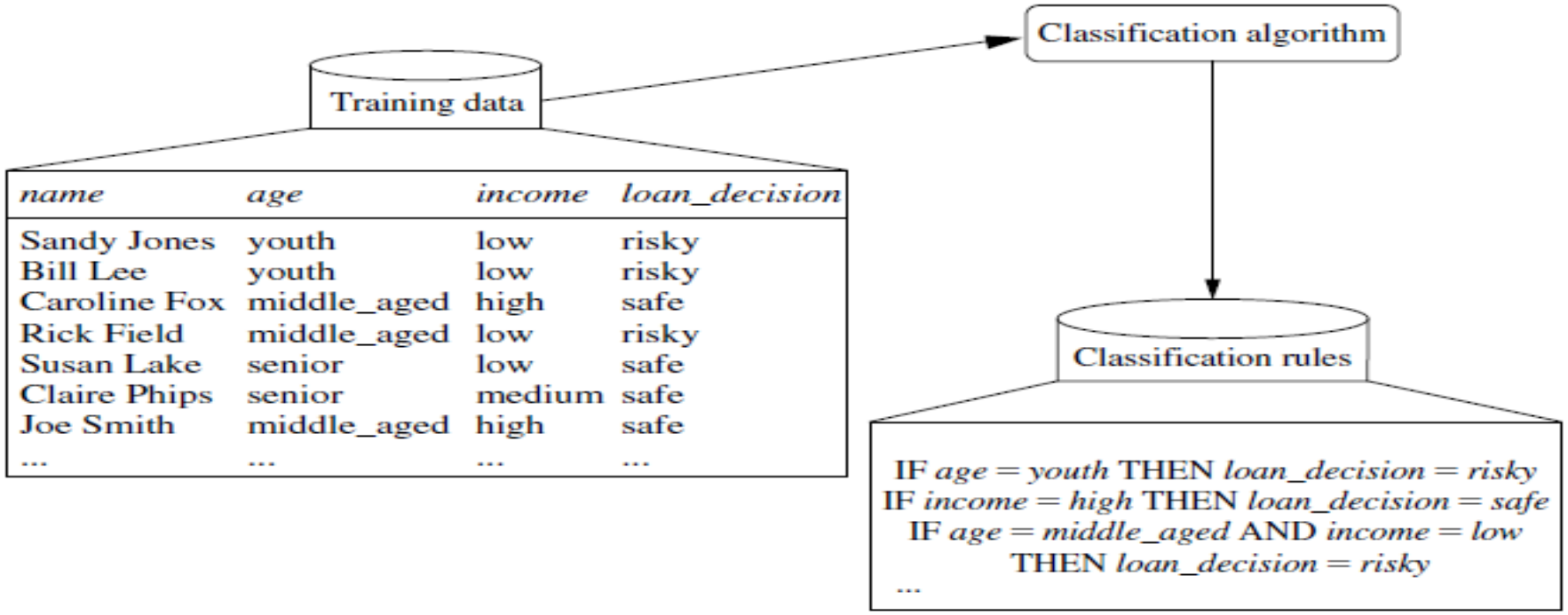
- Data classification is a two-step process, consisting of a:
  - Learning step: where a classification algorithm builds the classifier by analyzing or “learning from” a training set made up of database tuples and their associated class labels.
  - Classification step: where the model is used to predict class labels for given data.

General Approach to Classification

1. **Model construction (learning step):** describing a set of predetermined classes
   - Each tuple/sample is assumed to belong to a predefined class, as determined by the *class label attribute*
   - The set of tuples used for model construction is *training set*
   - The model is represented as classification rules, decision trees, or mathematical formula.

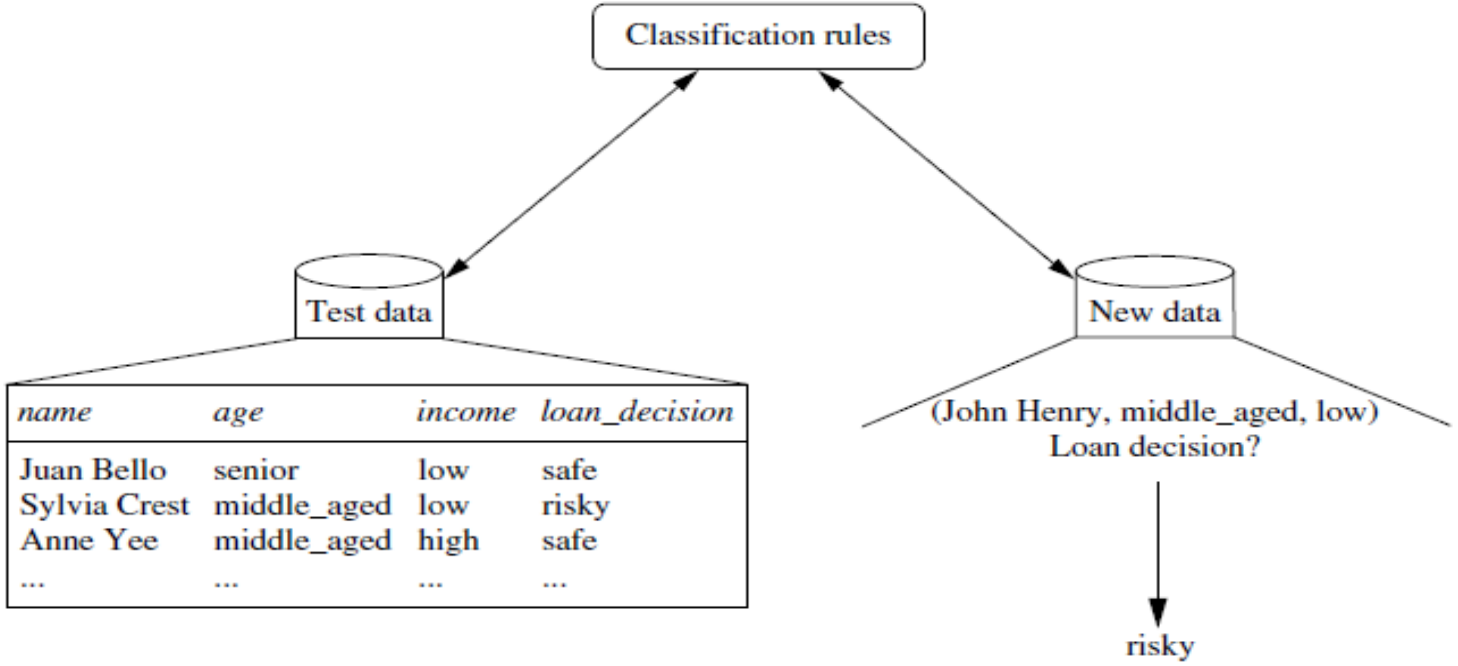
2. **Model usage (classification step):** for classifying future or unknown objects:
   - Estimate accuracy of the model
     - The known label of test sample is compared with the classified result from the model
     - Accuracy rate is the percentage of test set samples that are correctly classified by the model
   - If the accuracy is acceptable, use the model to classify new data

General Approach to Classification



Model construction (learning step)

General Approach to Classification



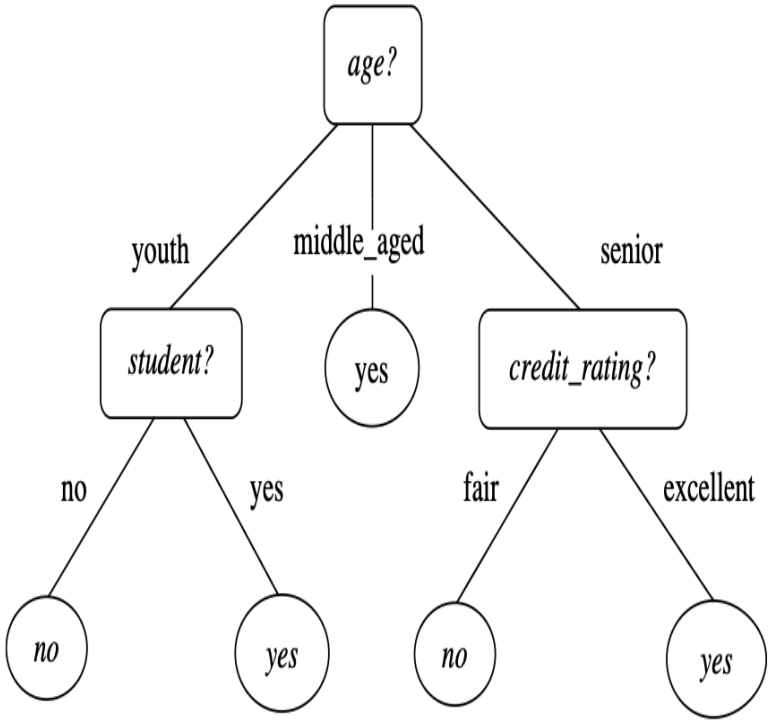
Model usage (classification step)

Classification Techniques

- Base Classifiers
  - Decision Tree based Methods
  - Rule-based Methods
  - Nearest-neighbor
  - Neural Networks
  - Deep Learning
  - Naïve Bayes and Bayesian Belief Networks
  - Support Vector Machines

Decision Tree Induction

- ***Decision tree induction*** is the learning of decision trees from class-labeled training tuples.
- A decision tree is a flowchart-like tree structure, where:
  - each internal node (non leaf node) denotes a test on an attribute,
  - each branch represents an outcome of the test,
  - each leaf node (or terminal node) holds a class label,
  - The topmost node in a tree is the root node.

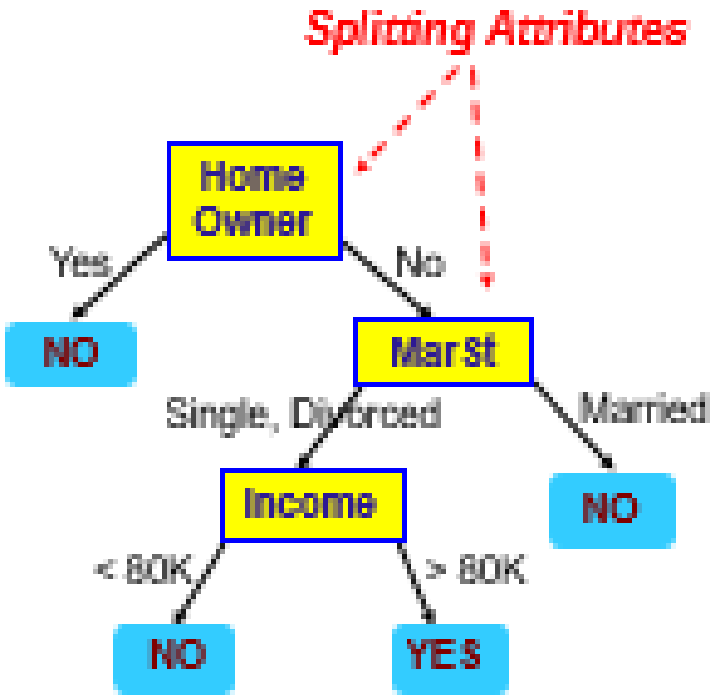


Decision Tree Induction: Example

categorical categorical continuous class

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

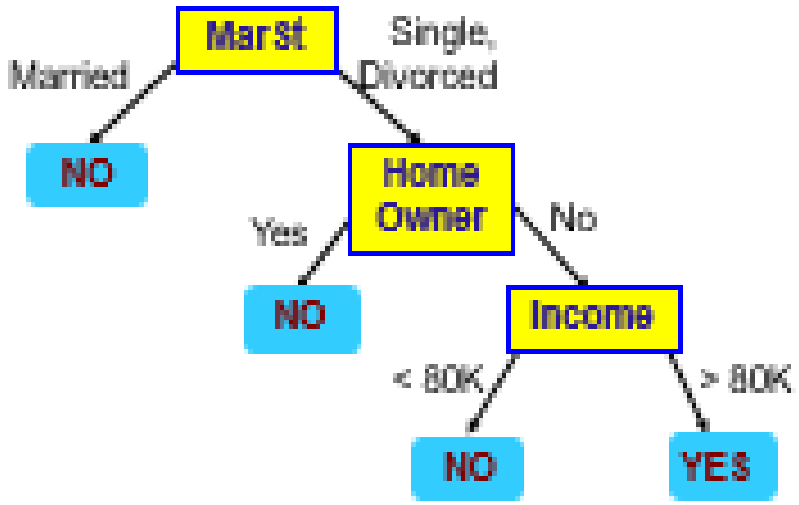
Training Data



Model: Decision Tree

Decision Tree Induction: Example

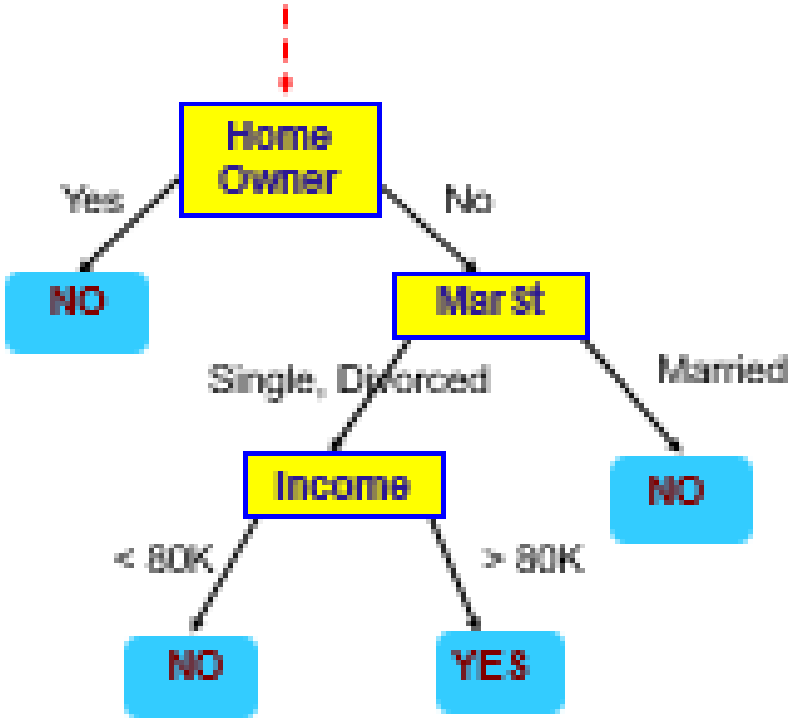
ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
		categorical	categorical	continuous	class
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



There could be more than one tree that fits the same data!

Apply Model to Test Data

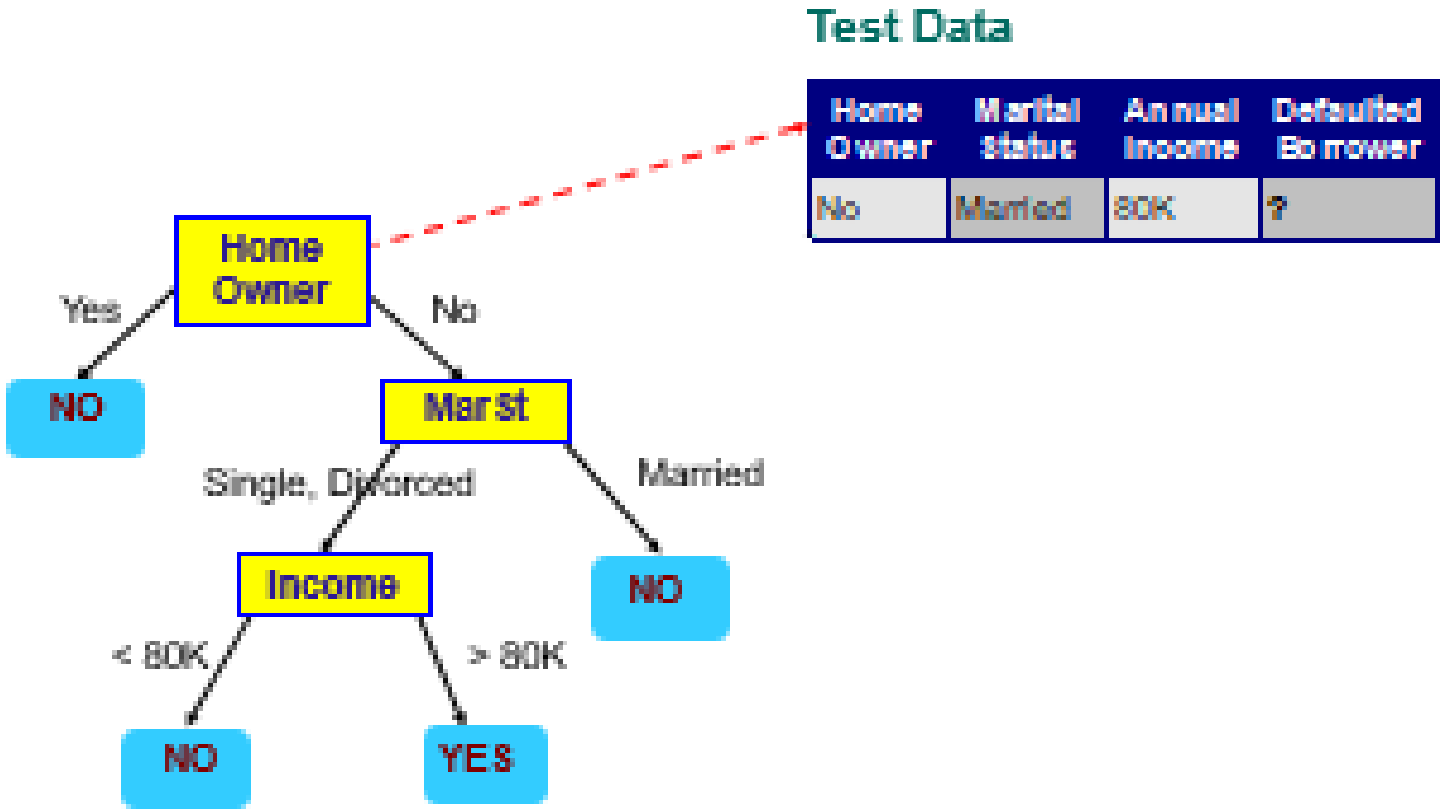
Start from the root of tree.



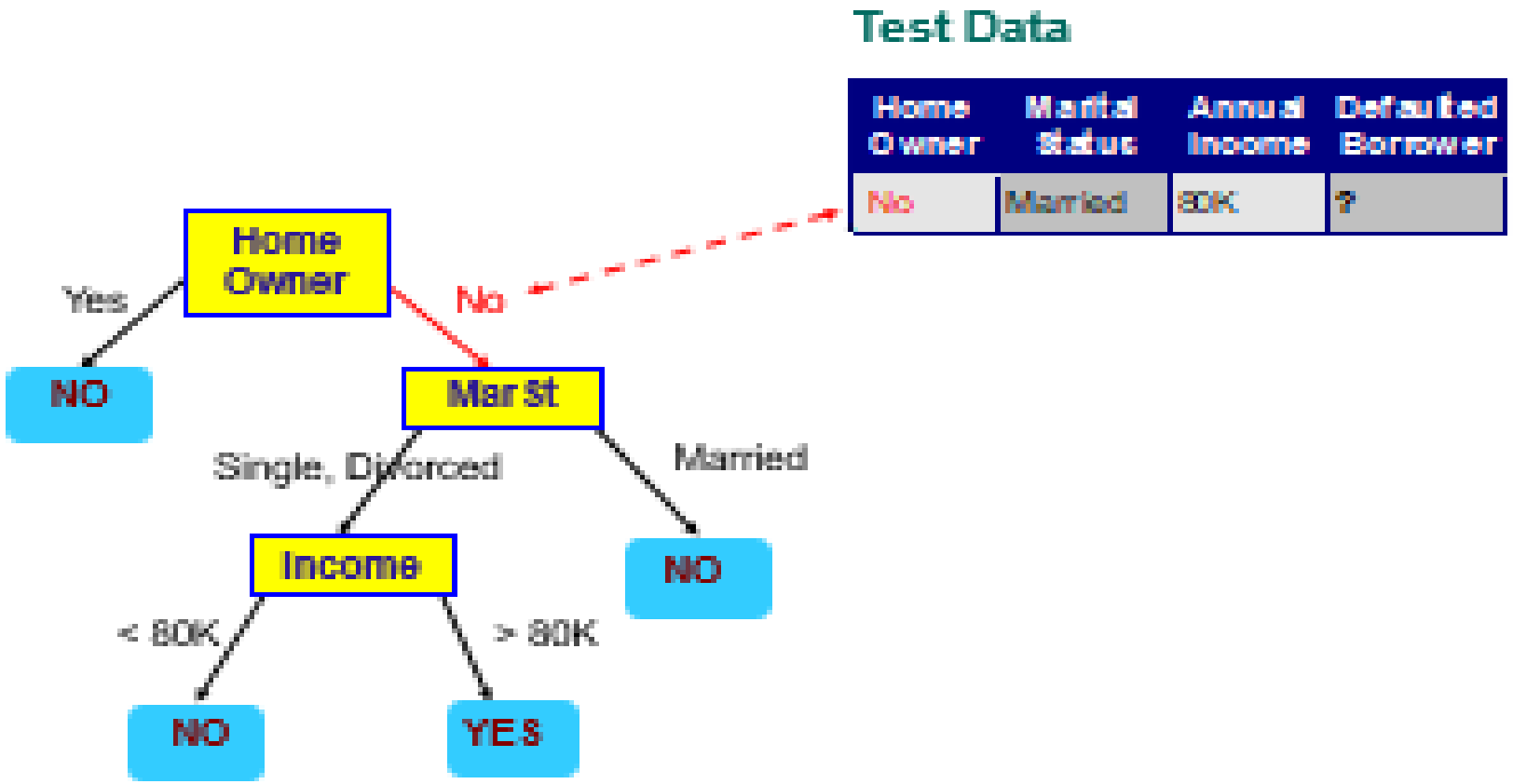
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

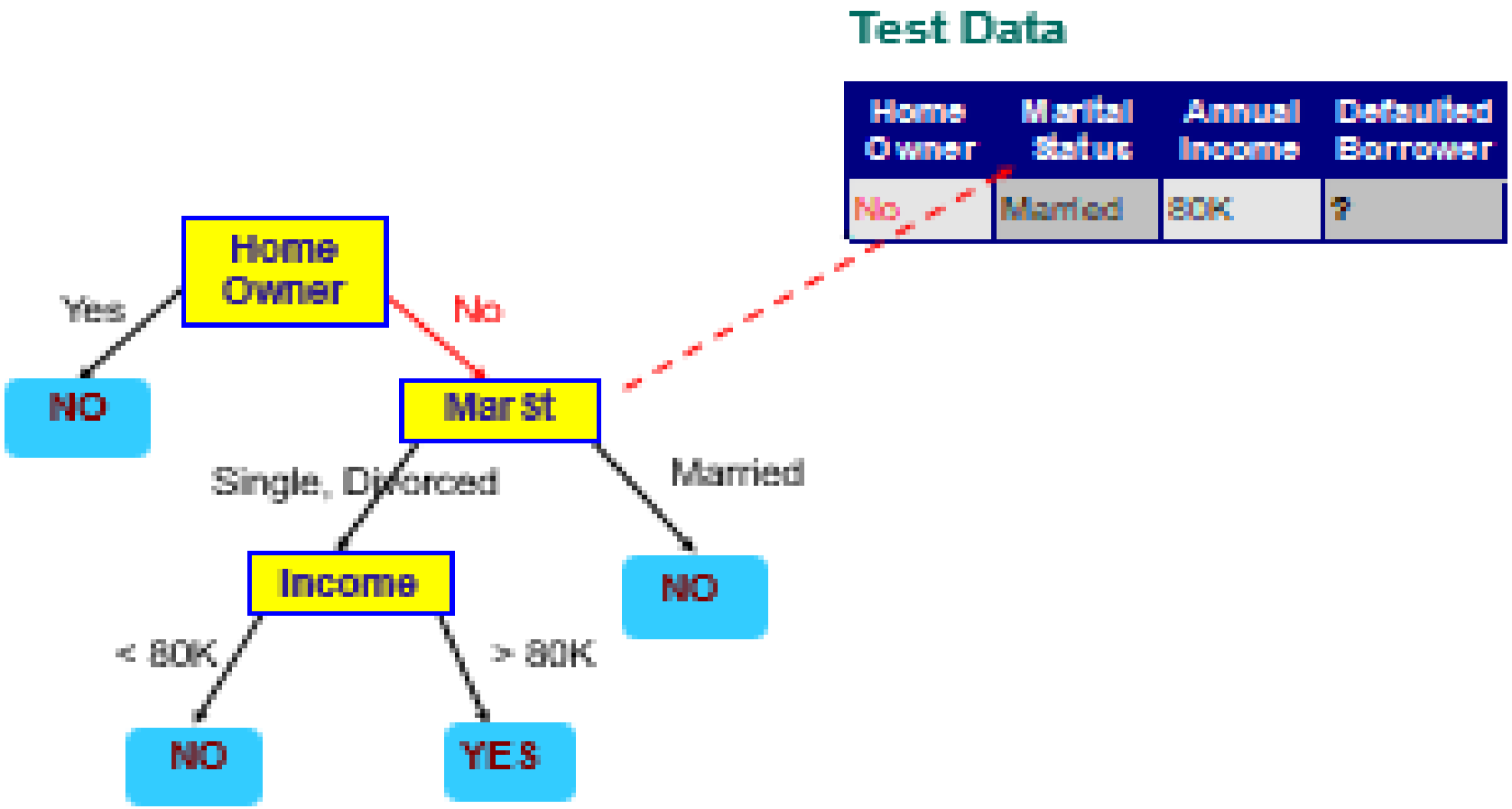
Apply Model to Test Data



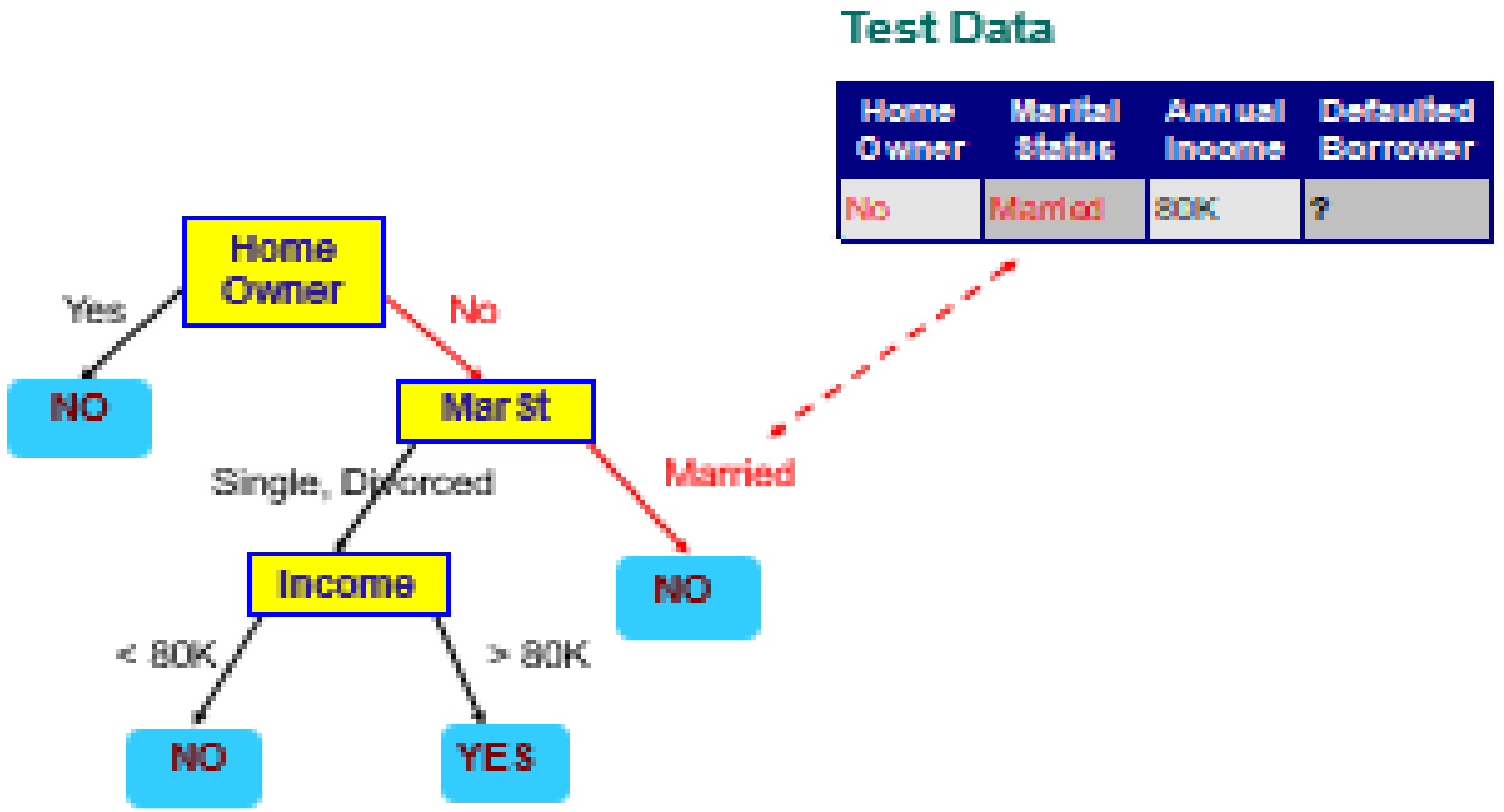
Apply Model to Test Data



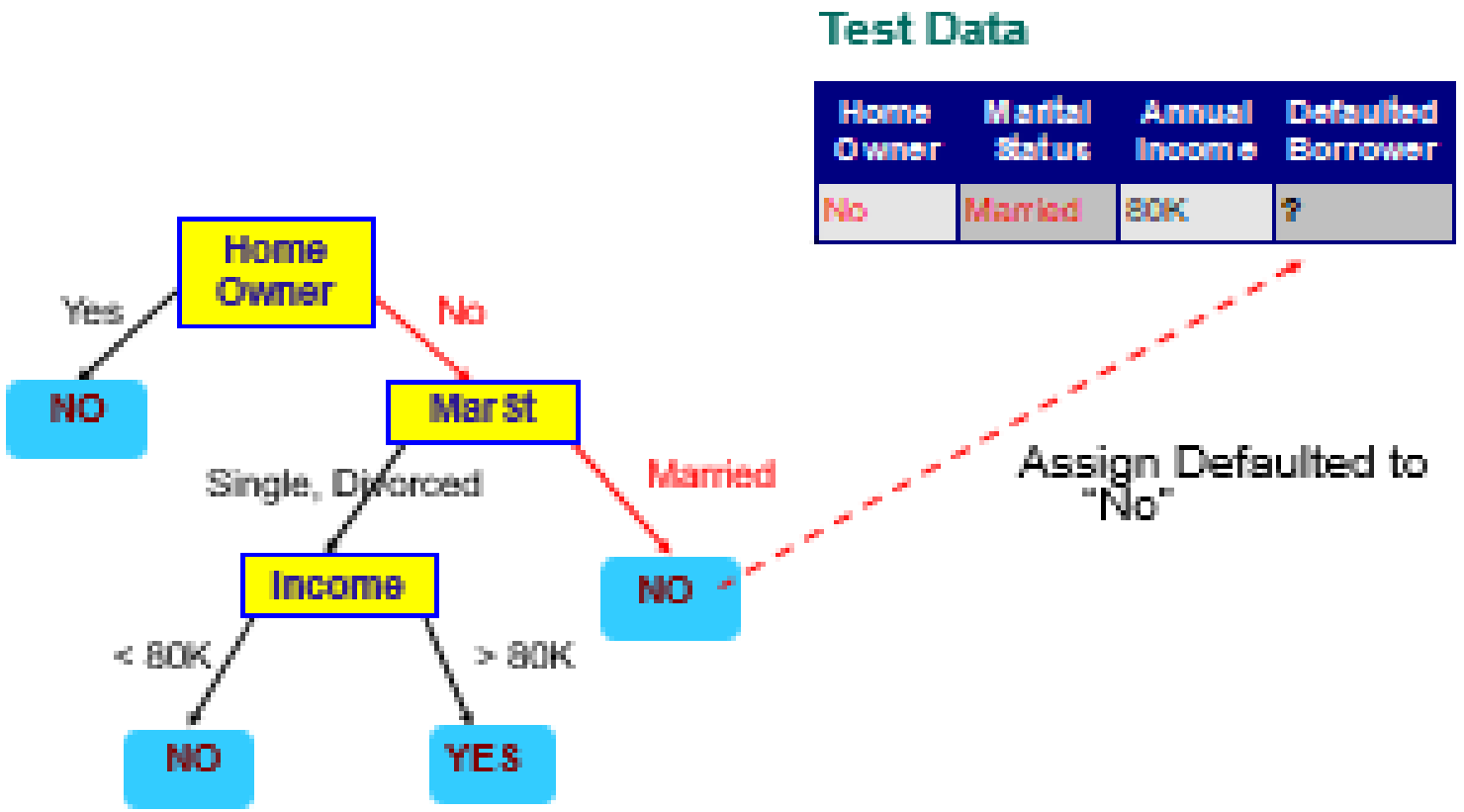
Apply Model to Test Data



Apply Model to Test Data



Apply Model to Test Data



Decision Tree Induction

Algorithm: Generate decision tree. Generate a decision tree from the training tuples of data partition, D.

- **Input**:
  - Data partition, D, which is a set of training tuples and their associated class labels;
  - Attribute list, the set of candidate attributes;
  - Attribute selection method, a procedure to determine the splitting criterion that "best" partitions the data tuples into individual classes. This criterion consists of a splitting attribute and, possibly, either a split-point or splitting subset.
- **Output:** A decision tree.

Decision Tree Induction

Basic algorithm for inducing a decision tree

```
(1)  create a node N;
(2)  if tuples in D are all of the same class, C, then
(3)      return N as a leaf node labeled with the class C;
(4)  if attribute_list is empty then
(5)      return N as a leaf node labeled with the majority class in D; // majority voting
(6)  apply Attribute_selection_method(D, attribute_list) to find the "best" splitting_criterion;
(7)  label node N with splitting_criterion;
(8)  if splitting_attribute is discrete-valued and
         multiway splits allowed then // not restricted to binary trees
(9)      attribute_list ← attribute_list − splitting_attribute; // remove splitting_attribute
(10) for each outcome j of splitting_criterion
     // partition the tuples and grow subtrees for each partition
(11)     let Dj be the set of data tuples in D satisfying outcome j; // a partition
(12)     if Dj is empty then
(13)         attach a leaf labeled with the majority class in D to node N;
(14)     else attach the node returned by Generate_decision_tree(Dj, attribute_list) to node N;
     endfor
(15) return N;
```

Decision Tree Induction

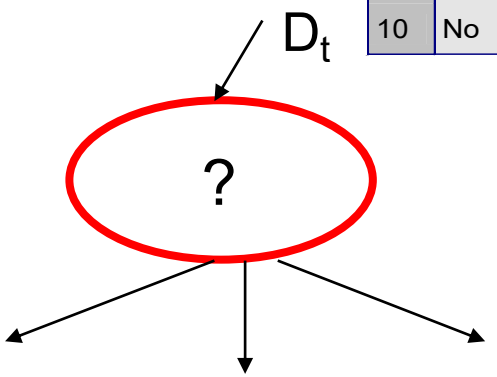
- Many Algorithms:
  - Hunt's Algorithm (one of the earliest)
  - CART (Classification and Regression Trees)
  - ID3 (Iterative Dichotomiser)
  - C4.5 (a successor of ID3)
  - SLIQ,SPRINT

Hunt's Algorithm: General Structure

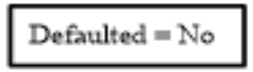
- Let D_t be the set of training records that reach a node t

- General Procedure:
  - If D_t contains records that belong the same class y_t, then t is a leaf node labeled as y_t
  - If D_t contains records that belong to more than one class, use an attribute test to split the data into smaller subsets. Recursively apply the procedure to each subset.

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



Hunt's Algorithm



(7,3)

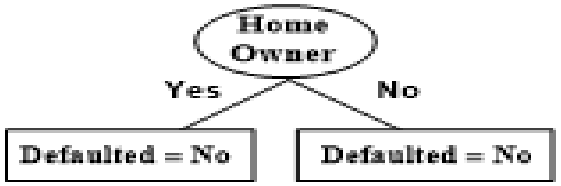
(a)

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Hunt's Algorithm

(7,3)

(a)

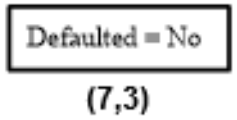


(3,0) (4,3)

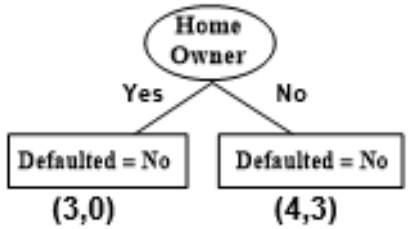
(b)

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

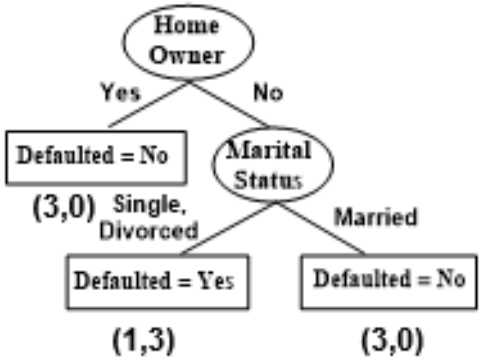
Hunt's Algorithm



(a)



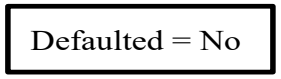
(b)



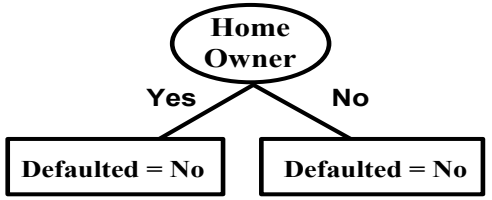
(c)

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

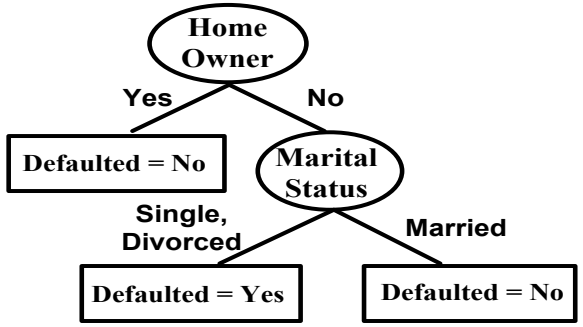
Hunt's Algorithm



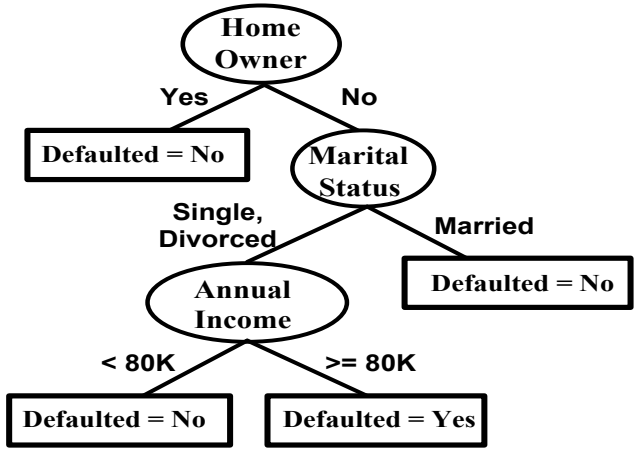
(a)



(b)



(c)



(d)

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Design Issues of Decision Tree Induction

- How should training records be split?
  - Method for specifying test condition
    - depending on attribute types
  - Measure for evaluating the goodness of a test condition

- How should the splitting procedure stop?
  - Stop splitting if all the records belong to the same class or have identical attribute values
  - Early termination

Attribute Selection Measures

An ***attribute selection measure*** is a heuristic for selecting the splitting criterion that "best" separates a given data partition, D, of class-labeled training tuples into individual classes.

Attribute selection measures:

- are known as splitting rules,
- provides a ranking for each attribute describing the given training tuples,
- the attribute having the best score for the measure is chosen as the splitting
- if we were to split D into smaller partitions according to the outcomes of the splitting criterion, ideally each partition would be pure.

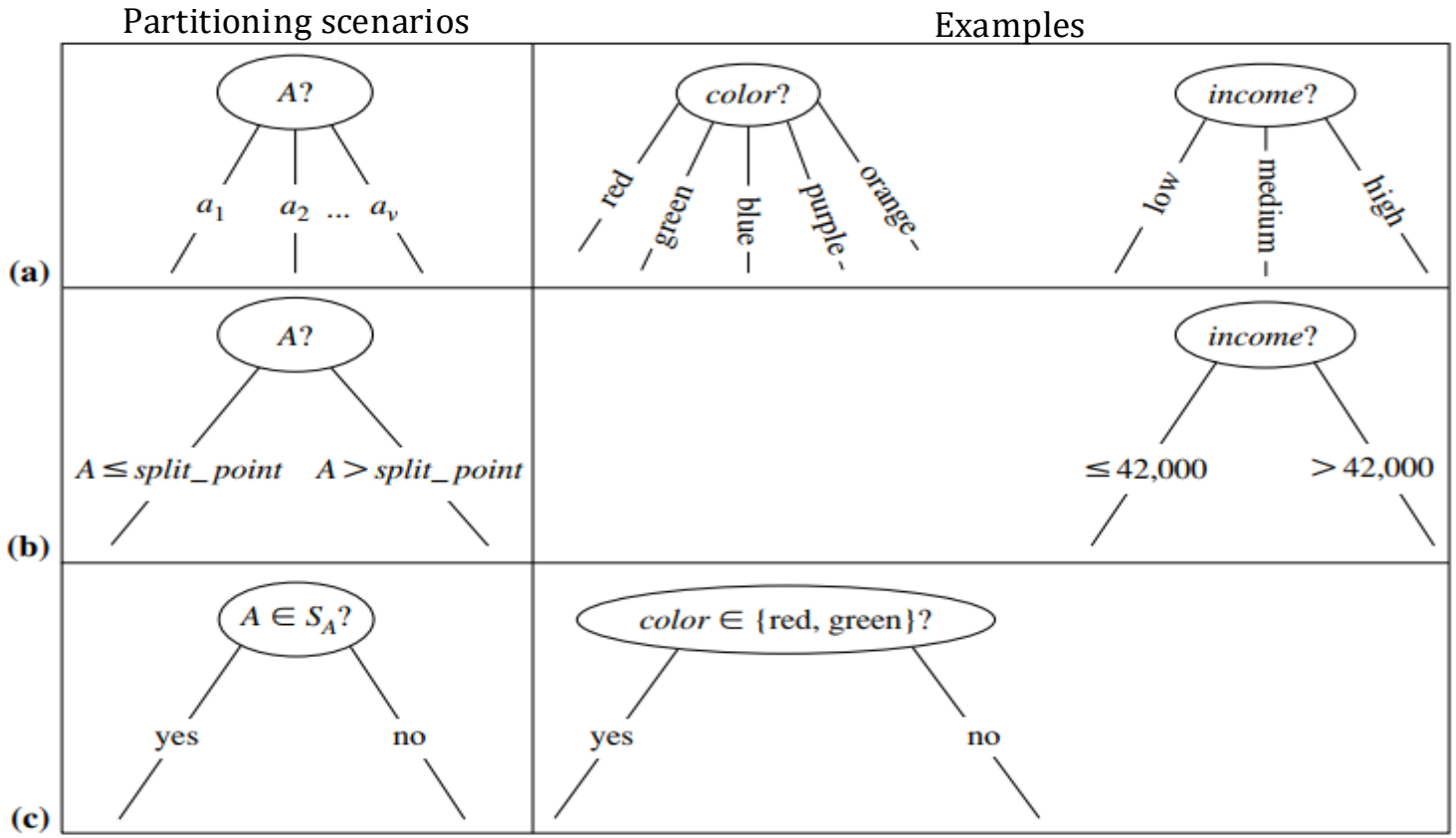
Attribute Selection Measures

- Depends on attribute types
  - Binary
  - Nominal
  - Ordinal
  - Continuous

- Depends on number of ways to split
  - 2-way split
  - Multi-way split

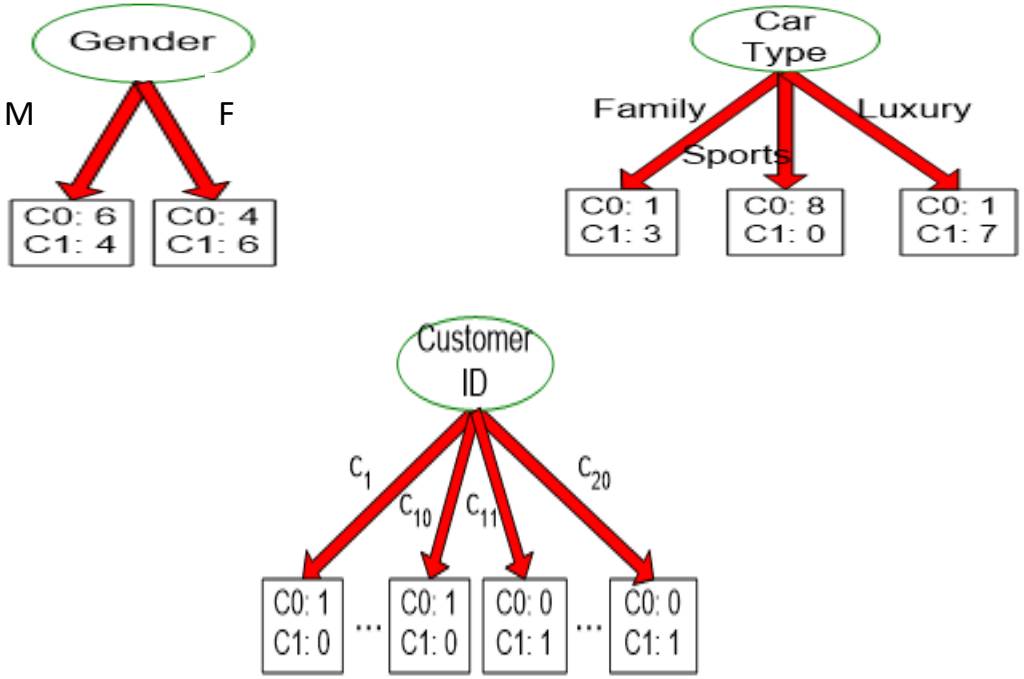
Attribute Selection Measures

Partitioning scenarios



Attribute Selection Measures

Before Splitting: 10 records of class 0,
10 records of class 1



Customer Id	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1

How to determine the Best Split ???

Attribute Selection Measures

- Greedy approach:
  - Nodes with purer class distribution are preferred
- Need a measure of node impurity:

Attribute Selection Measures

- Gini Index $$GINI(t) = 1 - \sum_j [p(j \mid t)]^2$$
- Entropy $$Entropy(t) = -\sum_j p(j \mid t) \log p(j \mid t)$$
- Misclassification error $$Error(t) = 1 - \max_i P(i \mid t)$$

Attribute Selection Measures

Finding the Best Split

1. Compute impurity measure (P) before splitting
2. Compute impurity measure (M) after splitting
   - Compute impurity measure of each child node
   - M is the weighted impurity of children
3. Choose the attribute test condition that produces the highest gain

$$\text{Gain} = P - M$$

or equivalently, lowest impurity measure after splitting (M)

Measure of Impurity: Gini Index

Gini Index for a single node t

$$GINI(t) = 1 - \sum_j [p(j \mid t)]^2$$

(NOTE: $p(j \mid t)$ is the relative frequency of class j at node t).

- Maximum $(1 - 1/n_c)$ when records are equally distributed among all classes, implying least interesting information
- Minimum (0.0) when all records belong to one class, implying most interesting information

Measure of Impurity: Gini Index

Gini Index for a single node t

$$GINI(t) = 1 - \sum_j [p(j \mid t)]^2$$

(NOTE: $p(j \mid t)$ is the relative frequency of class j at node t).

- For 2-class problem (p, 1 – p):
  - GINI = $1 - p^2 - (1 - p)^2 = 2p\ (1\text{-}p)$

C1	**0**
C2	**6**
Gini=0.000	

C1	**1**
C2	**5**
Gini=0.278	

C1	**2**
C2	**4**
Gini=0.444	

C1	**3**
C2	**3**
Gini=0.500	

Measure of Impurity: Gini Index

Gini Index for a single node t

C1	**0**
C2	**6**

$P(C1) = 0/6 = 0 \quad P(C2) = 6/6 = 1$

$Gini = 1 - P(C1)^2 - P(C2)^2 = 1 - 0 - 1 = 0$

C1	**1**
C2	**5**

$P(C1) = 1/6 \quad P(C2) = 5/6$

$Gini = 1 - (1/6)^2 - (5/6)^2 = 0.278$

clearly 0 is the best

C1	**2**
C2	**4**

$P(C1) = 2/6 \quad P(C2) = 4/6$

$Gini = 1 - (2/6)^2 - (4/6)^2 = 0.444$

Measure of Impurity: Gini Index

Gini Index for a collection of nodes

- When a node p is split into k partitions (children)

$$GINI_{split} = \sum_{i=1}^{k} \frac{n_i}{n} GINI(i)$$

where, n_i = number of records at child i,
n = number of records at parent node p.

- Choose the attribute that minimizes weighted average Gini index of the children

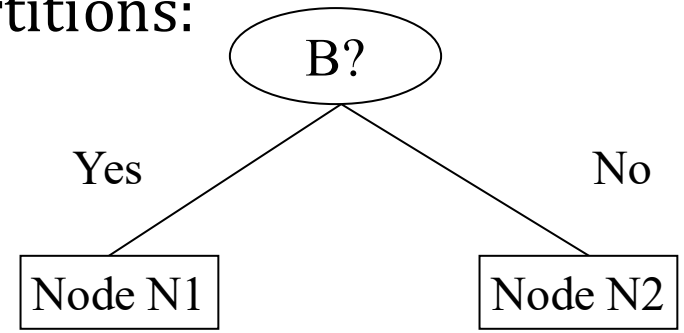
- Gini index is used in decision tree algorithms such as CART, SLIQ, SPRINT

Measure of Impurity: Gini Index

Gini Index for a collection of nodes

$$GINI_{split} = \sum_{i=1}^{k} \frac{n_i}{n} GINI(i)$$

- Splits into two partitions
- Effect of Weighing partitions:



	Parent
C1	7
C2	5
Gini = 0.486	

Gini(N1)
= 1 – $(5/6)^2$ – $(1/6)^2$
= 0.278

Gini(N2)
= 1 – $(2/6)^2$ – $(4/6)^2$
= 0.444

	N1	N2
C1	5	2
C2	1	4
Gini=0.361		

Weighted Gini of N1 N2
= 6/12 * 0.278 +
6/12 * 0.444
= 0.361

Gain = 0.486 – 0.361 = 0.125

Measure of Impurity: Gini Index

- For each distinct value, gather counts for each class in the dataset
- Use the count matrix to make decisions

Multi-way split

	CarType		
	Family	Sports	Luxury
C1	1	8	1
C2	3	0	7
Gini	0.163		

Two-way split
(find best partition of values)

	CarType	
	{Sports, Luxury}	{Family}
C1	9	1
C2	7	3
Gini	0.468	

	CarType	
	{Sports}	{Family, Luxury}
C1	8	2
C2	0	10
Gini	0.167	

Which of these is the best?

Measure of Impurity: Entropy

Entropy at a given node t

$$Entropy(t) = -\sum_j p(j \mid t) \log_2 p(j \mid t)$$

- (NOTE: *p(j | t)* is the relative frequency of class j at node t).

- ***Entropy*** is **0** if all the members of the collection belong to the same class.
- ***Entropy*** is **1** when the collection contains an equal no. of positive and negative examples.
- ***Entropy*** is between **0** and **1** if the collection contains unequal no. of positive and negative examples.

Measure of Impurity: Entropy

Entropy at a given node t

$$Entropy(t) = -\sum_j p(j \mid t) \log_2 p(j \mid t)$$

C1	**0**
C2	**6**

P(C1) = 0/6 = 0 P(C2) = 6/6 = 1

Entropy = – 0 log 0 – 1 log 1 = – 0 – 0 = 0

C1	**1**
C2	**5**

P(C1) = 1/6 P(C2) = 5/6

Entropy = – (1/6) $\log_2$ (1/6) – (5/6) $\log_2$ (1/6) = 0.65

clearly 0 is the best

C1	**2**
C2	**4**

P(C1) = 2/6 P(C2) = 4/6

Entropy = – (2/6) $\log_2$ (2/6) – (4/6) $\log_2$ (4/6) = 0.92

Measure of Impurity: Entropy

Computing Information Gain After Splitting

$$GAIN_{split} = Entropy(p) - \left(\sum_{i=1}^{k} \frac{n_i}{n} Entropy(i) \right)$$

- Parent Node, p is split into k partitions;
  n_i is number of records in partition i

- Choose the split that achieves most reduction (maximizes GAIN)

- Used in the ID3 and C4.5 decision tree algorithms

Measure of Impurity: Classification Error

- **Classification error at a node t**

$$Error(t) = 1 - \max_i P(i \mid t)$$

  - Maximum $(1 - 1/n_c)$ when records are equally distributed among all classes, implying least interesting information

  - Minimum (0) when all records belong to one class, implying most interesting information

Measure of Impurity: Classification Error

Classification error at a node t

$$Error(t) = 1 - \max_i P(i \mid t)$$

C1	**0**
C2	**6**

P(C1) = 0/6 = 0 P(C2) = 6/6 = 1

Error = 1 – max (0, 1) = 1 – 1 = 0

C1	**1**
C2	**5**

P(C1) = 1/6 P(C2) = 5/6

Error = 1 – max (1/6, 5/6) = 1 – 5/6 = 1/6

C1	**2**
C2	**4**

P(C1) = 2/6 P(C2) = 4/6

Error = 1 – max (2/6, 4/6) = 1 – 4/6 = 1/3

clearly 0 is the best

Outlines

- Classification: Basic Concepts
- Decision Tree Induction
- Bayes Classification Methods 
- Rule-Based Classification
- Model Evaluation and Selection
- Summary

Bayesian Classification: Why?

- A statistical classifier: performs *probabilistic prediction, i.e.,* predicts class membership probabilities,
- Foundation: Based on Bayes' Theorem,
- Performance: A simple Bayesian classifier, *naïve Bayesian classifier*, has comparable performance with decision tree and selected neural network classifiers

Naïve Bayesian Classifier

Determine the most probable class label for each object

- Based on the observed object attributes
  - Naïvely assumed to be conditionally independent of each other
- Example:
  - Based on the objects attributes {shape, color, weight}
  - A given object that is {spherical, yellow, < 60 grams}, may be classified (labeled) as a tennis ball
- Class label probabilities are determined using Bayes' Law

Input variables are discrete

Output:

- Probability score – proportional to the true probability
- Class label – based on the highest probability score

Naïve Bayesian Classifier - Use Cases

Preferred method for many text classification problems.

- Try this first; if it doesn't work, try something more complicated

Use cases

- Spam filtering, other text classification tasks
- Fraud detection

Building a Training Dataset to Predict Good or Bad Credit

Predict the credit behavior of a credit card applicant from applicant's attributes:

- Personal status
- Job type
- Housing type
- Savings amount

These are all categorical variables and are better suited to Naïve Bayesian Classifier.

personal_status	job	housing	savings_status	credit_class
male single	skilled	own	no known savings	good
female div/dep/mar	skilled	own	<100	bad
male single	unskilled resident	own	<100	good
male single	skilled	for free	<100	good
male single	skilled	for free	<100	bad
male single	unskilled resident	for free	no known savings	good
male single	skilled	own	500<=X<1000	good
male single	high qualif/self emp/mgm	rent	<100	good
male div/sep	unskilled resident	own	>=1000	good
male mar/wid	high qualif/self emp/mgm	own	<100	bad
female div/dep/mar	skilled	rent	<100	bad
female div/dep/mar	skilled	rent	<100	bad
female div/dep/mar	skilled	own	<100	good
male single	unskilled resident	own	<100	bad
female div/dep/mar	skilled	rent	<100	good
female div/dep/mar	unskilled resident	own	100<=X<500	bad
male single	skilled	own	no known savings	good
male single	skilled	own	no known savings	good
female div/dep/mar	high qualif/self emp/mgm	for free	<100	bad
male single	skilled	own	500<=X<1000	good
male single	skilled	own	<100	good
male single	skilled	rent	500<=X<1000	good
male single	unskilled resident	rent	<100	good
male single	skilled	own	100<=X<500	good
male mar/wid	skilled	own	no known savings	good
male single	unskilled resident	own	<100	good
male mar/wid	unskilled resident	own	<100	good

Technical Description - Bayes' Law

$$P(C \mid A) = \frac{P(A \cap C)}{P(A)} = \frac{P(A \mid C)P(C)}{P(A)}$$


Reverend Thomas Bayes

C is the class label:

- $C \in \{C_1, C_2, \ldots C_n\}$

A is the observed object attributes

- $A = (a_1, a_2, \ldots a_m)$

P(C | A) is the probability of C given A is observed

- Called the conditional probability

Apply the Naïve Assumption and Remove a Constant

- For observed attributes A = (a_1, a_2, … a_m), we calculate

$$P(C_i \mid A) = \frac{P(a_1, a_2, \ldots, a_m \mid C_i) P(C_i)}{P(a_1, a_2, \ldots, a_m)} \qquad i = 1, 2, \ldots, n$$

  and assign the classifier, C_i , with the largest $P(C_i|A)$

- Two simplifications to the calculations
  - Apply naïve assumption - each a_j is conditionally independent of each other, then

$$P(a_1, a_2, \ldots, a_m \mid C_i) = P(a_1 \mid C_i) P(a_2 \mid C_i) \cdots P(a_m \mid C_i) = \prod_{j=1}^{m} P(a_j \mid C_i)$$

  - Denominator $P(a_1, a_2, \ldots a_m)$ is a constant and can be ignored

Building a Naïve Bayesian Classifier

Applying the two simplifications

$$P(C_i \mid a_1, a_2, \ldots, a_m) \propto \left(\prod_{j=1}^{m} P(a_j \mid C_i) \right) P(C_i) \qquad i = 1, 2, \ldots, n$$

To build a Naïve Bayesian Classifier, collect the following statistics from the training data:

- $P(C_i)$ for all the class labels.
- $P(a_j \mid C_i)$ for all possible a_j and C_i
- Assign the classifier label, C_i, that maximizes the value of

$$\left(\prod_{j=1}^{m} P(a_j \mid C_i) \right) P(C_i) \qquad i = 1, 2, \ldots, n$$

Naïve Bayesian Classifiers for the Credit Example

Class labels: {good, bad}

- P(good) = 0.7
- P(bad) = 0.3

Conditional Probabilities

- P(own|bad) = 0.62
- P(own|good) = 0.75
- P(rent|bad) = 0.23
- P(rent|good) = 0.14
- … and so on

personal_status	job	housing	savings_status	credit_class
male single	skilled	own	no known savings	good
female div/dep/mar	skilled	own	<100	bad
male single	unskilled resident	own	<100	good
male single	skilled	for free	<100	good
male single	skilled	for free	<100	bad
male single	unskilled resident	for free	no known savings	good
male single	skilled	own	500<=X<1000	good
male single	high qualif/self emp/mgm	rent	<100	good
male div/sep	unskilled resident	own	>=1000	good
male mar/wid	high qualif/self emp/mgm	own	<100	bad
female div/dep/mar	skilled	rent	<100	bad
female div/dep/mar	skilled	rent	<100	bad
female div/dep/mar	skilled	own	<100	good
male single	unskilled resident	own	<100	bad
female div/dep/mar	skilled	rent	<100	good
female div/dep/mar	unskilled resident	own	100<=X<500	bad
male single	skilled	own	no known savings	good
male single	skilled	own	no known savings	good
female div/dep/mar	high qualif/self emp/mgm	for free	<100	bad
male single	skilled	own	500<=X<1000	good
male single	skilled	own	<100	good
male single	skilled	rent	500<=X<1000	good
male single	unskilled resident	rent	<100	good
male single	skilled	own	100<=X<500	good
male mar/wid	skilled	own	no known savings	good
male single	unskilled resident	own	<100	good
male mar/wid	unskilled resident	own	<100	good

Naïve Bayesian Classifier for a Particular Applicant

Given applicant attributes of
A= {female single,
owns home,
self-employed,
savings > $1000}

Since P(good|A) > (bad|A), assign the applicant the label "good" credit

a_j	C_i	$P(a_j \mid C_i)$
female single	good	0.28
female single	bad	0.36
own	good	0.75
own	bad	0.62
self emp	good	0.14
self emp	bad	0.17
savings>1K	good	0.06
savings>1K	bad	0.02

P(good|A) ~ (0.28*0.75*0.14*0.06)*0.7 = 0.0012

P(bad|A) ~ (0.36*0.62*0.17*0.02)*0.3 = 0.0002

Naïve Bayes Classifier: Example 2

Class:
C1:buys_computer = 'yes'
C2:buys_computer = 'no'

Data to be classified:
X = (age <=30,
Income = medium,
Student = yes
Credit_rating = Fair)

age	income	student	credit_rating	buys_comp
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Naïve Bayes Classifier: An Example

- $P(C_i)$: P(buys_computer = "yes") = 9/14 = 0.643
  P(buys_computer = "no") = 5/14= 0.357
- Calculate $P(X|C_i)$ for each class
  P(age = "<=30" | buys_computer = "yes") = 2/9 = 0.222
  P(age = "<= 30" | buys_computer = "no") = 3/5 = 0.6
  P(income = "medium" | buys_computer = "yes") = 4/9 = 0.444
  P(income = "medium" | buys_computer = "no") = 2/5 = 0.4
  P(student = "yes" | buys_computer = "yes) = 6/9 = 0.667
  P(student = "yes" | buys_computer = "no") = 1/5 = 0.2
  P(credit_rating = "fair" | buys_computer = "yes") = 6/9 = 0.667
  P(credit_rating = "fair" | buys_computer = "no") = 2/5 = 0.4
- **X = (age <= 30 , income = medium, student = yes, credit_rating = fair)**

$P(X|C_i)$: P(X|buys_computer = "yes") = 0.222 x 0.444 x 0.667 x 0.667 = 0.044
P(X|buys_computer = "no") = 0.6 x 0.4 x 0.2 x 0.4 = 0.019

$P(X|C_i)*P(C_i)$: P(X|buys_computer = "yes") * P(buys_computer = "yes") = 0.028
P(X|buys_computer = "no") * P(buys_computer = "no") = 0.007

Therefore, X belongs to class ("buys_computer = yes")

age	income	student	credit_rating	buys_comp
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Naïve Bayes Classifier: Comments

- Advantages
  - Easy to implement
  - Good results obtained in most of the cases
- Disadvantages
  - Assumption: class conditional independence, therefore loss of accuracy
  - Practically, dependencies exist among variables
    - E.g., hospitals: patients: Profile: age, family history, etc.
      Symptoms: fever, cough etc., Disease: lung cancer, diabetes, etc.
    - Dependencies among these cannot be modeled by Naïve Bayes Classifier

Outlines

- Classification: Basic Concepts
- Decision Tree Induction
- Bayes Classification Methods
- Rule-Based Classification
- Model Evaluation and Selection
- Summary

Using IF-THEN Rules for Classification

- Represent the knowledge in the form of IF-THEN rules

  R: IF *age* = youth AND *student* = yes THEN *buys_computer* = yes

- Assessment of a rule: *coverage* and *accuracy*
  - n_{covers} = # of tuples covered by R
  - $n_{correct}$ = # of tuples correctly classified by R

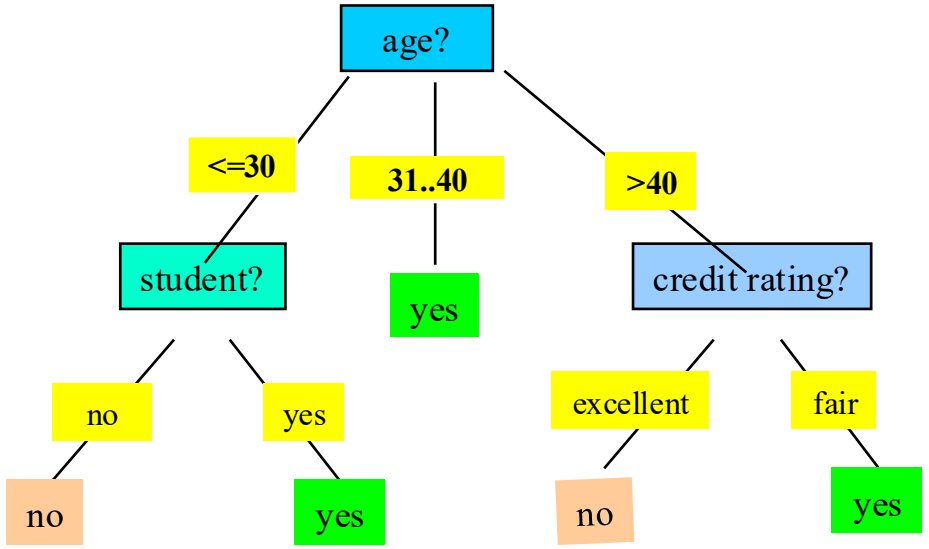
  coverage(R) = n_{covers} /|D| /* D: training data set */

  accuracy(R) = $n_{correct}$ / n_{covers}

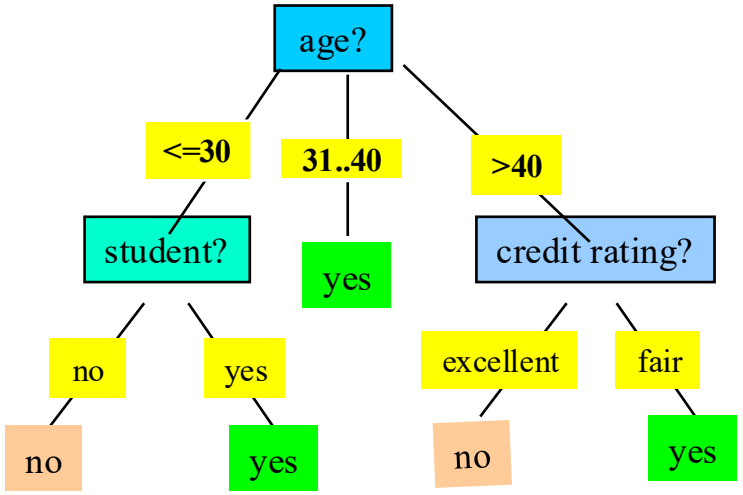
62

Rule Extraction from a Decision Tree

- Rules are *easier to understand* than large trees
- One rule is created *for each path* from the root to a leaf
- Each attribute-value pair along a path forms a conjunction: the leaf holds the class prediction
- Rules are mutually exclusive and exhaustive



Rule Extraction from a Decision Tree



- Rule extraction from our *buys_computer* decision-tree

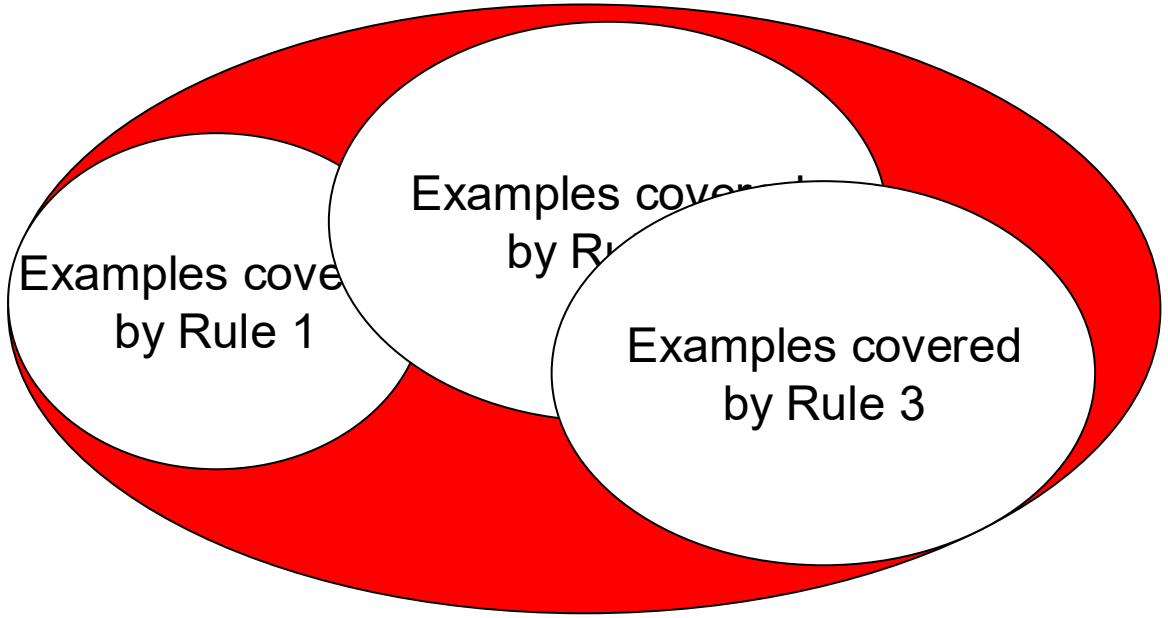
  IF *age* = young AND *student* = *no* THEN *buys_computer* = *no*
  IF *age* = young AND *student* = *yes* THEN *buys_computer* = *yes*
  IF *age* = mid-age THEN *buys_computer* = *yes*
  IF *age* = old AND *credit_rating* = *excellent* THEN *buys_computer* = *no*
  IF *age* = old AND *credit_rating* = *fair* THEN *buys_computer* = *yes*

Rule Induction: Sequential Covering Method

- Sequential covering algorithm: Extracts rules directly from training data
- Typical sequential covering algorithms: FOIL, AQ, CN2, RIPPER
- Rules are learned *sequentially*, each for a given class C_i will cover many tuples of C_i but none (or few) of the tuples of other classes
- Steps:
  - Rules are learned one at a time
  - Each time a rule is learned, the tuples covered by the rules are removed
  - Repeat the process on the remaining tuples until *termination condition*, e.g., when no more training examples or when the quality of a rule returned is below a user-specified threshold

Sequential Covering Algorithm

while (enough target tuples left)
    generate a rule
    remove positive target tuples satisfying this rule



Rule Generation

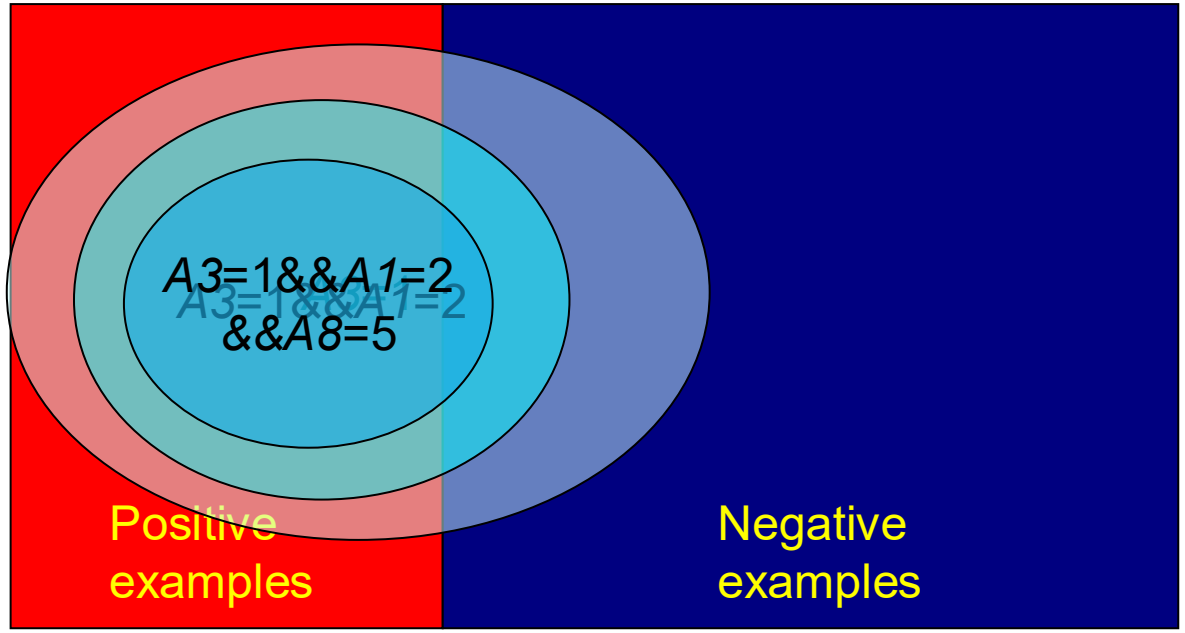
- To generate a rule

  while(true)

    find the best predicate *p*

    if foil-gain(*p*) > threshold **then** add *p* to current rule

    else break



Outlines

- Classification: Basic Concepts
- Decision Tree Induction
- Bayes Classification Methods
- Rule-Based Classification
- Model Evaluation and Selection 
- Summary

Model Evaluation and Selection

- Evaluation metrics: How can we measure accuracy? Other metrics to consider?
- Use **validation test set** of class-labeled tuples instead of training set when assessing accuracy
- Methods for estimating a classifier's accuracy:
  - Holdout method, random subsampling
  - Cross-validation
  - Bootstrap

Classifier Evaluation Metrics: Confusion Matrix

Confusion Matrix:

Actual class\Predicted class	C_1	$\neg C_1$
C_1	**True Positives (TP)**	**False Negatives (FN)**
$\neg C_1$	**False Positives (FP)**	**True Negatives (TN)**

Example of Confusion Matrix:

Actual class\Predicted class	buy_computer = yes	buy_computer = no	Total
buy_computer = yes	**6954**	**46**	7000
buy_computer = no	**412**	**2588**	3000
Total	7366	2634	10000

- Given *m* classes, an entry, $\boldsymbol{CM_{i,j}}$ in a **confusion matrix** indicates # of tuples in class *i* that were labeled by the classifier as class *j*
- May have extra rows/columns to provide totals

Classifier Evaluation Metrics: Accuracy, Error Rate, Sensitivity and Specificity

A\P	C	¬C	
C	**TP**	**FN**	**P**
¬C	**FP**	**TN**	**N**
	P'	**N'**	**All**

- **Classifier Accuracy,** or recognition rate: percentage of test set tuples that are correctly classified

  Accuracy = (TP + TN)/All
- **Error rate:** *1 – accuracy*, or

  Error rate = (FP + FN)/All

- **Class Imbalance Problem**:
  - **Sensitivity**: True Positive recognition rate
    - **Sensitivity = TP/P**
  - **Specificity**: True Negative recognition rate
    - **Specificity = TN/N**

Classifier Evaluation Metrics: Precision and Recall, and F-measures

- **Precision**: exactness – what % of tuples that the classifier labeled as positive are actually positive

$$precision = \frac{TP}{TP + FP}$$

- **Recall:** completeness – what % of positive tuples did the classifier label as positive?
  - Perfect score is 1.0
  - Inverse relationship between precision & recall

$$recall = \frac{TP}{TP + FN}$$

- ***F* measure (*F_1* or *F*-score)**: harmonic mean of precision and recall,

$$F = \frac{2 \times precision \times recall}{precision + recall}$$

Classifier Evaluation Metrics: Example

Actual Class\Predicted class	cancer = yes	cancer = no	Total	Recognition(%)
cancer = yes	**90**	**210**	300	30.00 (*sensitivity*
cancer = no	**140**	**9560**	9700	98.56 (*specificity)*
Total	230	9770	10000	96.40 (*accuracy*)

- *Precision* = 90/230 = 39.13% *Recall* = 90/300 = 30.00%

Outlines

- Classification: Basic Concepts
- Decision Tree Induction
- Bayes Classification Methods
- Rule-Based Classification
- Model Evaluation and Selection
- Summary

Summary

- Classification is a form of data analysis that extracts models describing important data classes.
- Effective and scalable methods have been developed for decision tree induction, Naive Bayesian classification, rule-based classification.
- Evaluation metrics include: accuracy, sensitivity, specificity, precision, recall, F measure, and F_β measure.

Thank you